

A Novel instruction stream buffer for VLIW architectures

Jih-Ching Chiu, Kai-Ming Yang*

Department of Electrical Engineering, National Sun Yat-Sen University, 70 Lien-hai Rd., Kaohsiung 804, Taiwan

ARTICLE INFO

Article history:

Received 13 January 2008

Received in revised form 11 May 2009

Accepted 26 August 2009

Available online 27 September 2009

Keywords:

Instruction stream buffer

VLIW architecture

Instruction issue

Instruction fetcher

ABSTRACT

The instruction compression mechanism used to solve the drawbacks of traditional very long instruction word (VLIW) architectures often leads to poor code density in the instruction cache, which causes the irregular lengths of long instructions to cross the different cache line. These split long instructions cannot be fetched simultaneously, which creates a bottleneck for VLIW architectures. This paper proposes a buffering mechanism which can slide the split long instruction as a continuous form to offer better efficiency in instruction fetching. This approach helps maintain the behaviors of the software pipeline technology, which schedules iterative instructions to enhance the performance of streaming processing for VLIW architectures. In the proposed mechanism, the instruction stream buffer stores the repeat block completely and suspends as far as possible the cache access to reduce access time. The advantages of repeatedly issuing instructions in the instruction buffer and preventing split long instructions, can substantially improve the performance in fetching instructions. Simulation results show that the mechanism is efficient at the instruction level for the basic DSP/IMG library by improving performance by 35% on average.

© 2009 Elsevier Ltd. All rights reserved.

1. Introduction

The VLIW architectures are generally used in the DSP. However, structure hazards and data dependences result in poor utilization of the available parallelism. This indicates a quantity of NOP instructions must be filled in the instruction cache [1,2]. In the instruction cache, the poor code density indicates that decreases hit rate of access instructions or requires more silicon area of DSP subsystem. Hence, it is essential method that using the compressing instruction mechanism eliminates these unnecessary NOP (non operation) instructions. The Fig. 1 shows the compressing instruction mechanism that obviates NOP instruction and closely arranges instructions in cache line, called fetch packet. These same color instructions are called long instruction can be issued simultaneously without data dependence and structure hazard.

The compressing instruction mechanism brings another problem that called split fetch packet, which means irregular lengths of long instructions (varying from one to eight instructions) may be split across two different cache lines [3–5]. Beside, the split fetching-packet is usually occurred in the compressed instructions mechanism of the VLIW architectures. This problem causes the delays of fetching instructions in a program; and further, it will also affect the performance of VLIW architectures because these instructions in the split long instruction cannot be issue to execute unit simultaneously. Therefore, when VLIW architectures must execute repeatedly special operations, quantities of split fetch packets are not only appeared but the non-continuous behavior is causing great performance damage of fetching instructions, especially the particular purposes processor such as DSP.

Abbreviations: VLIW, very long instruction word; DSP, digital signal processor.

* Corresponding author.

E-mail addresses: chiujhc@ee.nsysu.edu.tw (J.-C. Chiu), d953010024@student.nsysu.edu.tw (K.-M. Yang).

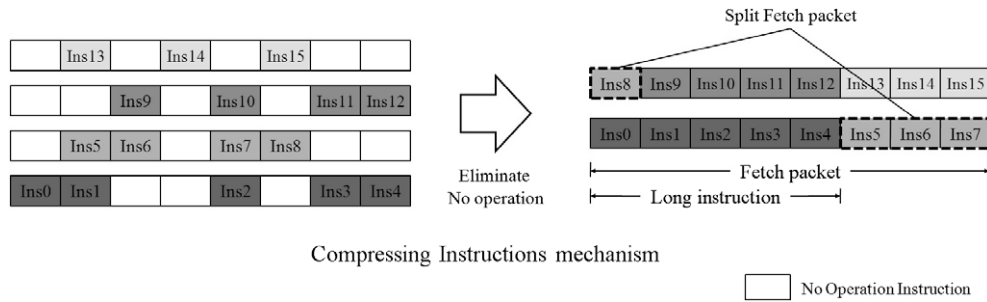


Fig. 1. Compressing instructions mechanism bring about split fetch packet. The left graph indicate that instruction cache are filled with NOP instructions as the structure hazards and data dependences. The right graph indicates that the compressing instructions mechanism eliminate NOP instruction to save the space of instruction cache.

To improve performance, the zero overhead hardware loops is implemented in the DSP because DSP activity is often characterized by many repetitions of few fixed algorithms. Those special operations are usually executed repeatedly during most parts of digital signal operations such as the FFT with lots of butterfly operations. Nevertheless, the problem, many repetitions follows split fetch packet, exist still in the DSP for VLIW architectures. If the instruction fetcher cannot keep a good performance, the efficiency of DSP will not be improved efficiently. According to Amdahl's Law [6], it is important to increase the efficiency of the instruction fetcher [7].

This paper starts with an overview of current compressing instructions method. Section 3 will introduce the architecture and mechanism of instruction stream buffer. The chapter 4 shows the verification and the result of simulation. Finally, the conclusion will be described in the chapter 5.

2. Previous work

2.1. Overview of instruction packet for current VLIW architecture

Because of structure hazards and data dependences, the instructions cannot always fill every computing unit in the VLIW architecture. The current VLIW processors offer the different method of instruction packet. This section will describe the method of instruction packet for more known VLIW processors such as Intel IA-64, TI C64x and TigerSHARC.

To solve the poor code density, IA-64 instructions encoded in bundles are the fixed number of instructions. Each bundle consists of a 5-bit template field and three instructions, each 41 bits in length. The 5-bit template filed with each bundle describes 24 possible template values. Every template indicates the instruction slots and stops for format. However, the arrangement of instruction type and the parallelism of instructions will be fixed in the 24 possible templates. This instruction bundle in IA-64 limits the arrangement of instructions so reduces the ILP (instruction-level parallelism). The method could cause the bottleneck of performance [6].

In the TI C64x DSP, it adds a 2-level on-chip cache; thereby the purpose in this architecture is to hit the most of the memory accesses in the smaller first level cache [4]. In addition, this architecture constituted by two clusters and uses the larger register file to decrease the number of data memory accesses [2]. Each cluster comprehends four function units and can transfer the data of register to another with cross data path. These imply that the most ILP is eight. In the method of instructions packet, the C64x is similar to TigerSHARC. This mechanism will be discussed in the next section [4,8,9].

The TigerSHARC processor core contains two compute blocks similar to the clusters in C64x DSP. Each block contains a register file and four independent computation units. And, improving ILP each compute block is facilitated by the ability to execute up to four 32-bit wide instructions per cycle. The TigerSHARC processor uses a variation of static schedule architecture to allow the programmer to specify which instructions are executed in parallel in each cycle. The instructions do not have to be aligned in memory so that program memory is not wasted.

In the summary, whatever C64x or TigerSHARC, they can flexibly arrange the instruction to improve ILP because the instructions, which can be issued to execute stage at the same time, are not limited by several specific templates such as IA-64. The C64x and TigerSHARC DSP can obtain the better ILP.

2.2. Compressing Instruction mechanism

To achieve the better ILP, the C64x and TigerSHARC DSP add the LEB (Long instruction Ending Bit) in the instruction format. The Fig. 2 is an example how to determine a long instruction by LEB in fetch packet. The execution of the individual instructions is partially controlled by LEB in each instruction. In the Fig. 2, the LEB are scanned from left to right. The LEB determines whether the instruction executes in parallel with another instruction. If the LEB of instruction ins2 is 1, then instruction ins3 is to be executed in parallel with (in the same cycle as) instruction ins2. If the LEB of instruction ins0 is 0, then instruction ins1 is executed in the cycle after instruction ins0. In the sample of Fig. 2, the only ins0 can be issued

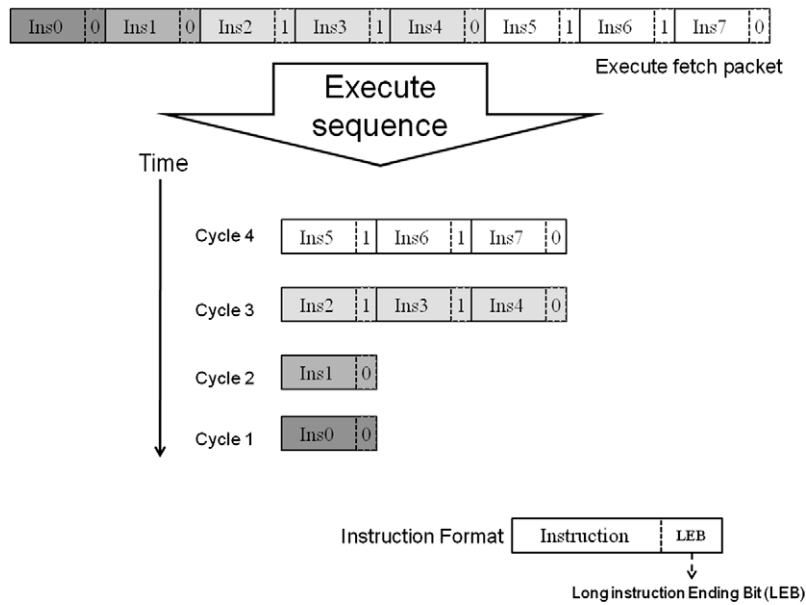


Fig. 2. Partially Serial LEB Pattern in a Fetch Packet. The LEB determent whether the front of instructions are executed simultaneously. The executing sequence of fetch packet is from left to right. In the cycle 1, the only ins0 can be executed. In the cycle 3, the ins2, ins3 and ins4 can be executed at the same time.

in the first cycle. Because the LEB of ins2, ins3 and ins4 are “110”, the ins2-ins4 can be issued simultaneously in the third cycle.

Because the number of instructions in the long instruction are variably, the compressing instruction mechanism above method bring about the split fetch packet. However, every fetch packet almost follows the split fetch packet. In this paper, a buffing mechanism proposes which can link the split long instruction to offer the better efficiency of instruction fetcher.

In this paper, the novel instruction stream buffer for VLIW Architectures mechanism is proposed which can link the split long instruction to improve the performance of issuing instruction. Beside, the loop body of software pipeline can be maintained in the buffer. Therefore, the instructions of loop body can be issued from the proposed buffer during loop execution, which prevent stall cycle due to wait the access cycle from program memory [10,13,14].

3. The buffering technology

This section will describe the architecture, hardware design of instruction stream buffer and buffering mechanism. The buffering technology improves the efficiency of fetching instruction in the split fetching-packet case. Besides, this section will brief describe how to handle the repeat instructions fetched in the instruction stream buffer.

3.1. The architecture of instruction stream buffer

This section will briefly describe the hardware architecture. This architecture of instruction stream buffer is shown in Fig. 3. The instruction stream buffer contains three parts: (1) pre-fetch buffer (2) operating buffer constructed by buffer1 and buffer2 and (3) instruction stream buffer controller.

The size of buffer1, buffer2 and pre-fetch buffer are as one fetch packet length each. The pre-fetch buffer can ahead fetch next fetch packet after loop body, preventing stall cycles due to miss instruction as multi-cycles access.

The operating buffer, buffer1 and buffer2, makes up a rotatable buffer (operating buffer). The instructions in buffer2 can be shift to the buffer1. And, the source of buffer2 may be from buffer1, program memory or pre-fetch buffer by the instruction stream buffer controller. Specially, according to the current length of issuing long instruction in buffer1, controller will determine when the buffer2 fetch next fetch packet.

The LEB in the instruction of buffer1 from left to right determine the current long instruction which must be issued. These instructions in buffer1 can be shift to buffer2 in the loop body. And, the source of buffer1 may be from buffer2. Specially, if pre-fetch and operating buffer have not any instruction show step 1 in Fig. 5, the instructions from program memory will be bypass to next stage and prepare the remain instruction for shifting to buffer1 next cycle.

All instructions in operating buffer can be issue to next pipeline stage within one cycle. Beside, Buffer1 can select the fetching instruction source from program memory or buffer2 by Decoding Instruction Select (DIS). The fetching instruction sources of buffer2 are form buffer1 for keeping decoded instructions, pre-fetch buffer or program memory by Fetching

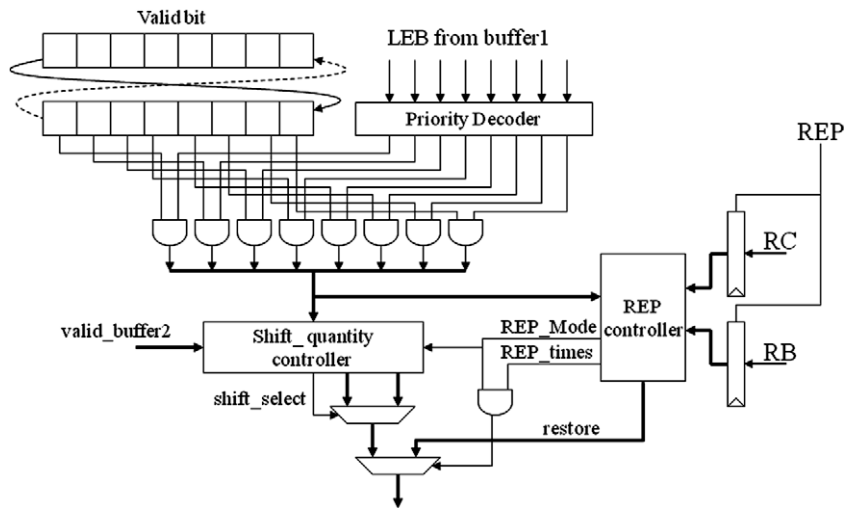


Fig. 3. The architecture of instruction stream buffer. The buffering mechanism architecture contains three parts: (1) pre-fetch buffer, (2) operating buffer, and (3) instruction stream buffer controller. The instruction stream buffer controller determines the shift offset for the operating buffer, the FIS selects the source of buffer2, and DIS selects the source of buffer1.

Instruction Select (FIS). The instruction stream buffer controller, show in Fig. 4, send DIS, FIS and shift offset to operate the operating buffer.

3.2. Novel instruction buffering mechanism

This section will prepare the buffering mechanism. The valid bit will be used to mark the valid instructions. So, the valid instructions will be defined below describe. The valid instructions mean the instructions must be executed but be not issued yet. In other words, the invalid instructions mean these instructions have been executed. For example, in the sequence case, the instruction11 is defined valid and the instruction9 is defining invalid when the long instruction10 is issuing. In the loop case, all instructions in loop body are valid before the end of loop. The same color instructions indicate a long instruction that can be issued simultaneously in below statement. The white indicate the invalid instructions.

The next sections will describe the buffering mechanism. This mechanism has two main active: (1) linking valid instructions in operating buffer. (2) Repeat block in operating buffer.

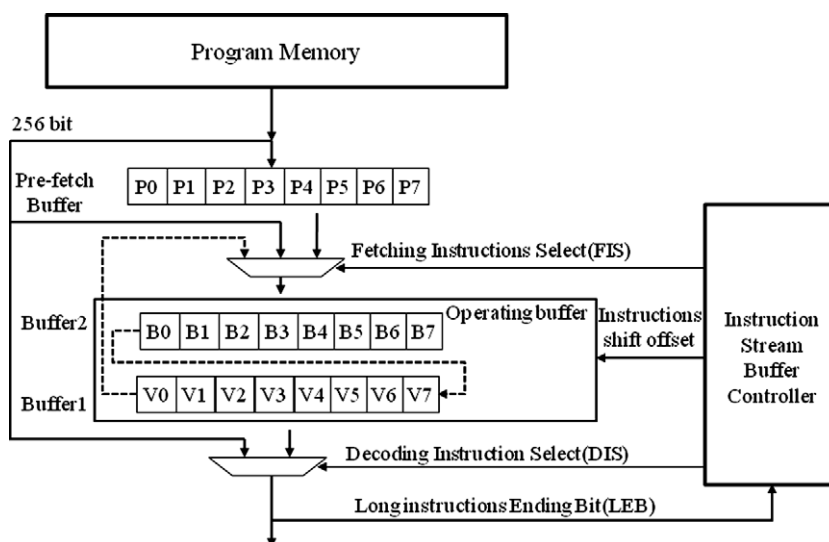


Fig. 4. The controller of Instruction stream buffer. Whether these instructions in the operating buffer are valid or invalid will be determined by the valid bit in the controller. The valid bit and LEB will be AND for shift offset. The REP controller controls the execution of repeat instructions. Finally, the correspondent offset will be sent to the operating buffer.

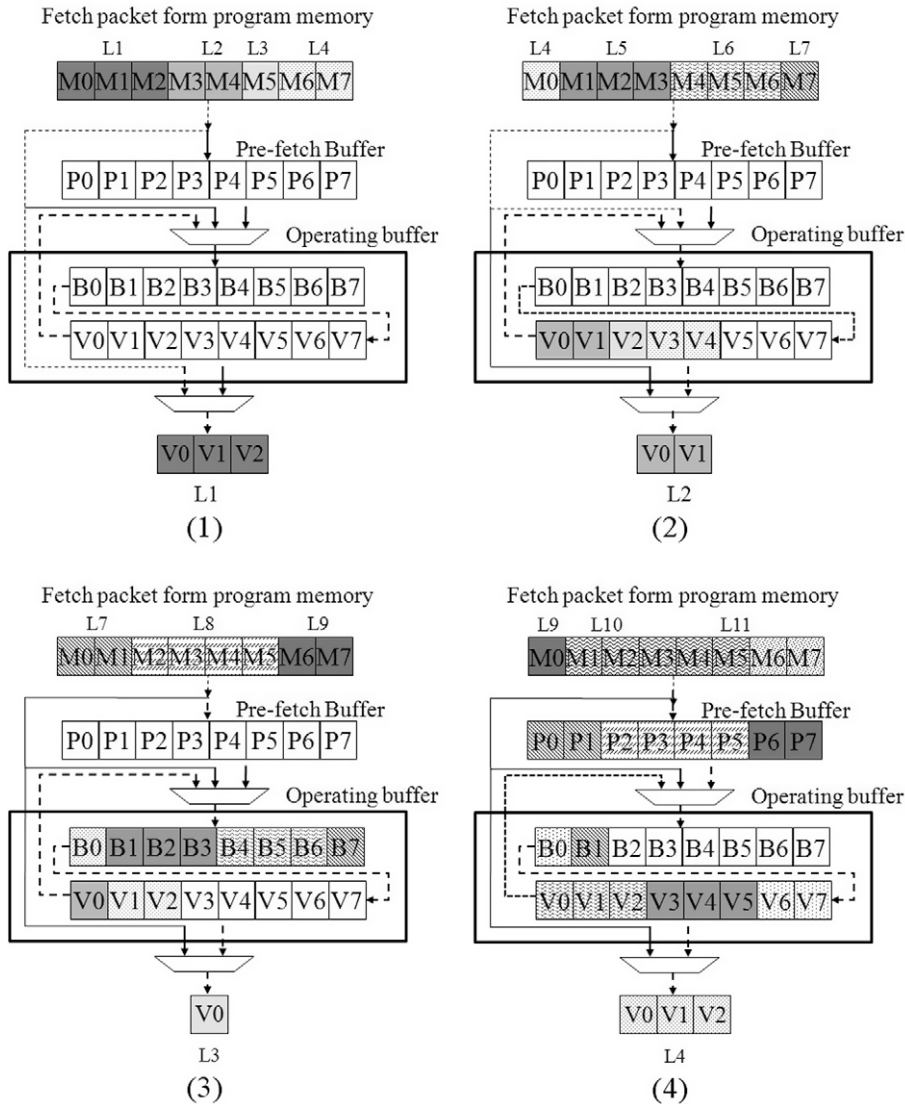


Fig. 5. Novel instruction buffering mechanism. An examples of the novel instruction buffering mechanism. The number below the picture is the step. In the initial condition, step1, all buffers are invalid. The fetch packet instructions come from the program memory. L1 is bypassed to avoid stalling the access cycle through the pre-fetch and operating buffer. Step 2 shifts the L2–L4 to buffer1 and selects a new fetch packet for buffer2. Step 3 shows that buffer1 shifts the distance of L2 and sends a new fetch packet to buffer2. Step4 show that the split long instruction is linked by the buffering mechanism.

3.2.1. Linking valid instructions in operating buffer

Fig. 5 is the split fetching-packet case. Step 1 indicates a state that is issuing L1 long instruction. Because pre-fetch and operating buffer have not any instruction in operating buffer, the instructions from program memory will be bypass to next stage and prepare the remain instruction for shifting to buffer1 next cycle. This design for bypass can reduce stall cycle of fetch packet from program memory to dispatch stage. The follow will illustrate the detail mechanism step by step.

- Step 1: To avoid the stall access cycle through the pre-fetch and operating buffer from program memory to next pipeline stage, the DIS can select the bypass path.
- Step 2: After step 1, the remaining valid instructions will be shift to bufer1 and the program memory will be selected by FIS to store the buffer2 in step2. And, the L2 will be issued and V5 ~ V7 become invalid instructions in buffer1 because the L1 have been issued in step1. Besides, the variable length of long instructions is distressful that every fetch packet almost follows the split fetch packet. The long instruction L4 is an example.
- Step 3: In step 3, long instruction L4 is split. The operating buffer can link L4 in step4 when the invalid instructions in the buffer1 are retained in step 3. This mechanism can combine the split long instruction in this step. Beside, the program memory will be access to the pre-fetch buffer in next cycle.

Step 4: In this step, the split long instruction was linked. Beside, the lengths of issuing long instruction (L4) are great than the valid long instruction in buffer2. It means the all instructions in buffer2 will be invalid in next cycle. Therefore, the controller of buffering mechanism controls FIS to select source of buffer2 from pre-fetch buffer. This mechanism can keep enough valid instructions in buffer2 to shift valid instruction for buffer1.

The problem of split fetch packet will become more serious in the repeated and considerable operations. Therefore, solving the split fetch packet is very important topic. The instruction stream buffer controller can send different shift offset (Cross I and Shift I) and control signal to buffer1 in Fig. 7. By the different shift offset, the operating buffer can cross invalid instructions to link valid instructions in Fig. 6.

3.2.2. Repeat block in operating buffer

The repeat instruction is usually used in the zero overhead loops. However, the non-continuous fetching of instructions faces a hazard that causes the difficulty for fetching instruction in repeat cases. The buffering mechanism not only combines the split fetch packet but keep the loop body in the operating buffer. This characteristic which retains the repeat block in buffer can be used to reduce the access from program memory. In other hand, if the all of long instructions in the repeat block are more numerous than the operating buffer, the repeat block cannot be retained in the operating buffer. The non-continuous instructions fetched must still access fetch packet from program memory.

This section will describe how to operate the behavior of repeat in the proposed mechanism. If the case of repeat block can be retained in operating buffer, the repeat instruction indicates the size of repeat block and the times of executing repeat block. This information will be stored in the RC (Repeat Count) and RB (Repeat Block) in the Fig. 4. When the repeat instruction is decoded, the valid bit will be held until the end of repeat times. In other words, we can get the times of repeat and the size of repeat block by a repeat instruction. Further, the RC is a register that count the times of repeat block. The size of repeat

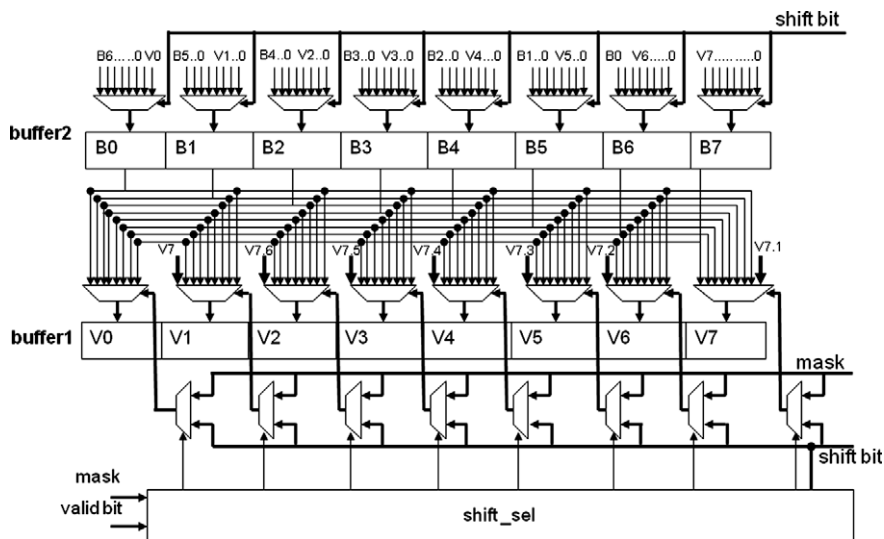


Fig. 6. Split fetches packet case. The V0–V1 shift the distance of L1. Other buffers shift the distance of L1 adds invalid instructions.

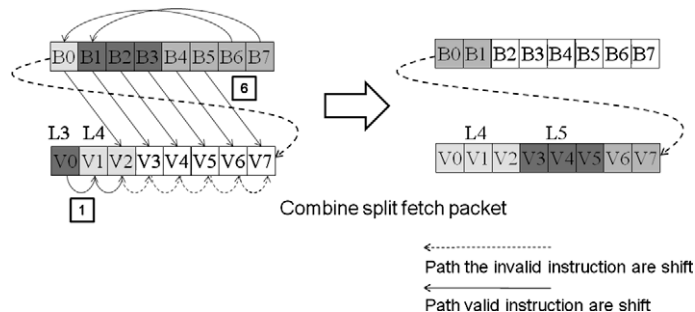


Fig. 7. Design of operating buffer to support different two offset.

block, recorded in the BR register determent when the instruction issue must back in the start of loop. If the size of repeat block is smaller than the operating buffer, the all repeat blocks will be retained in operating buffer until the RC register is zero. Besides, if repeat instructions are longer than the operating buffer, the repeat block cannot be maintained in the operating buffer. These instructions will be executed sequentially without retaining instructions. After the end of repeat block is executed, the mechanism will restart to fetch the start of repeat block.

3.3. Hardware design of instruction stream buffer

This section will describe the design of buffering mechanism to support operation of loop buffer and split fetch packet.

3.3.1. Design of buffering mechanism for loop operation

The Fig. 4 shows the design of instruction stream buffer controller. The valid bit record that the instructions in operating buffer is un-issued or issued. It determines whether the long instructions had been issued or not. After the long instructions are issued, the valid bit is recorded logic 0 (invalid), whereas logic 1. According to the valid bit and Long instruction End Bit (LEB), the instruction stream buffer controller will determine the offset for shift operating buffer and the DIS and FIS for instruction source. Moreover, when the repeat instruction is decoded, the shift offset and valid bit are determined by REP_controller to retain repeat block in operating buffer.

According to the shift offset, any long instructions in operating buffer can be rotated to decode within a fetching cycle. Hence the main principle of instruction buffering mechanism is to keep as possible long instructions in operating buffer that have been issued already. This buffering mechanism keeps loop body as possible in operating buffer to improve the performance of fetching long instructions.

3.3.2. Design of buffering mechanism for split fetch packet

This section will describe how to design the buffering for split fetch packet. The split fetch packet mean a long instruction is cross in different two cache line. The L4 in Fig. 5 is an example of split fetch packet. The buffering mechanism, shown in Fig. 7, supports different shift offset in the buffer1 to combine the split long instructions. After the L3 is issued, the operating buffer will shift an instruction to left. In Fig. 6, the V0 and V1 will obtain the L4 but V1–V6 acquire the invalid. For this reason, the design of buffer mechanism, shown in Fig. 7, support two shifts offset. The controller can calculate the shift offset “Cross I” and “shift I”. The shift I is the offset of valid instructions. The Cross I is the offset added the number of valid instructions and invalid instructions to cross the invalid instructions. The buffer1 can be select the Cross I or shift I. Hence, if instruction buffer will obtain the invalid instruction, the shift_sel select the Cross I to avoid the invalid instruction split the long instruction.

The proposed mechanism of instruction stream buffer offer two main improvement: (1) solving the split fetch packet. (2) Keep the repeat block in the operating buffer without accessing instructions from program memory. Especially, the software pipeline, used for VLIW architectures, can schedule an operation to repeat block kept in the operating buffer. Therefore, in the sequent instruction issue, the proposed mechanism solves the split fetch packet. In the repeated execution, the proposed mechanism not only links the split long instructions but cannot access instructions from program memory.

4. Simulation results

This chapter will describe the result of performance, simulation environment and estimation of hardware complexity.

4.1. Performance evaluation

According to Mibench, We evaluate the performance of the proposed instruction stream buffer through simulator for VLIW instruction fetcher. We select the common operations from TMS320C64x DSP and Image/Video Processing Library [11,12,15–17]. Then, we modeled the traditional VLIW instruction fetcher and proposed buffering instruction fetcher. Using these models simulated the fetch of instructions to obtain the split long instruction cycle. Beside, assuming that the cache miss rate is 0%, Fig. 8 shows the speedup after solving the split fetch packet.

In simulation result, the x-axis represents the operations of libraries, and the y-axis represents the speedup of buffering mechanism. The case of quantize and dotprod almost increase no performance due to the end of long instructions in loop body are just on the end of fetch packet exactly. Therefore, fetching long instructions in loop body will not have any split long instructions. However, the majority operation present still split fetch packet. The performance result of the average can be approximate 35% by the proposed buffering mechanism.

4.2. Hardware complexity

Our architecture of buffering mechanism is implemented by using Verilog and synthesized by Synopsys Design Compiler in a 0.18 μm technology. It's total area and critical path are 684641.64 μm^2 and 9.26 ns. In the buffering architecture, we design 3-pipe stage: (1) convert parallel bit (LEB) into shift offset. This stage consists of priority encoder. (2) Calculate the

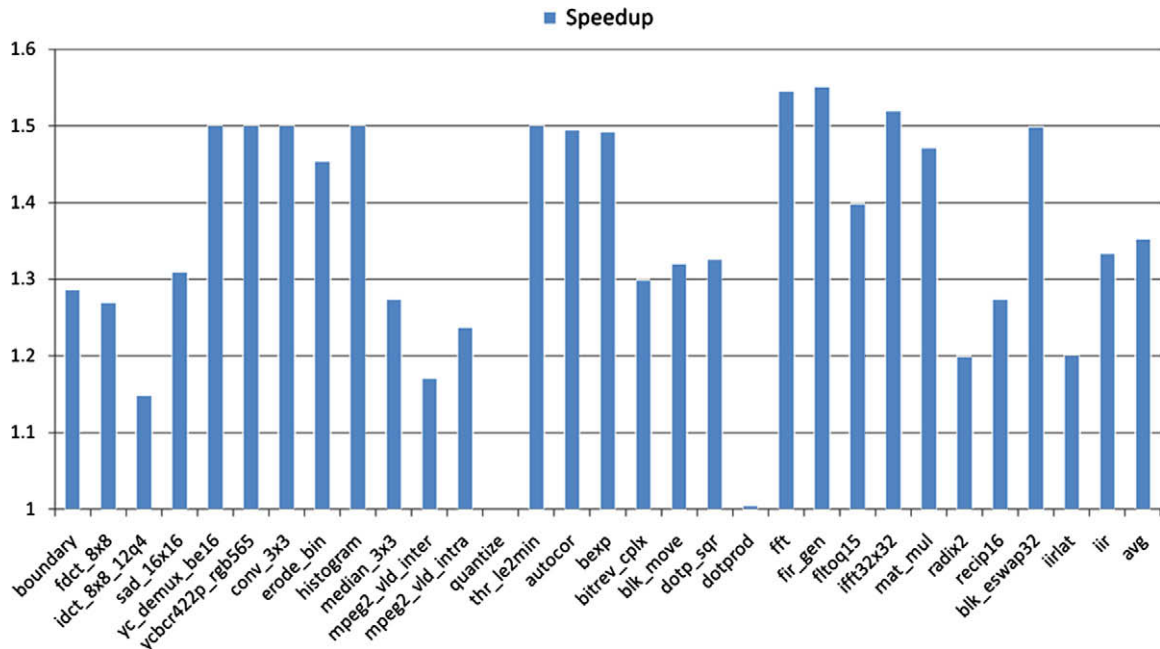


Fig. 8. Speedup of buffering mechanism.

shift offset. This stage consists of adder and multiplexer to calculate a quantity of shift offset in the operating buffer. (3) Select different the shift offset and shifts the operating buffer. In this stage, the 16–1 multiplexer is main part for rotate the operating buffer. Other part is the 2–1 multiplexer to the different two shift offset. The buffering mechanism can operate at frequency of up to 215 MHz. In the same 0.18 μm technology, the core area and speed, 4 instructions issue for VLIW architecture in [18], is approximate 3.23 mm $\times$ 3.23 mm and operate at 208 MHz. Therefore, the buffering mechanism, used in the same 0.18 μm technology, increases 6.6% area. Implementing the buffering mechanism in the 0.18 μm technology will not cause the bottleneck in speed.

5. Conclusions

In this paper, the novel instruction stream buffer for VLIW architectures is proposed. By the features of the proposed buffering mechanisms, the hazards of the split fetch packet in the compressed long instruction fetching can be solved and the hit rate of instruction fetching is improved for effectively keeping the repeat blocks in the operating buffer. By the simulation results, for TMS320C64x DSP and Image/Video Processing Library, the speedup can be improved up to 35%. The outcome in instruction level imports that the novel instruction stream buffer increases the approximate 5.7% area but avoids the split cache line and obtain 35% speedup in fetching instruction unit. The proposed mechanism will be suitable used in the variable length long instructions for VLIW architectures.

References

- [1] Larin SY, Conte TM. Compiler-driven cached code compression schemes for embedded ILP processors. In: IEEE international conference on computer design; November 1999. p. 82–92D.
- [2] Gass W. Higher performance and lower power enhancements to VLIW architectures. In: IEEE international conference on computer design; September 2001. p. 157.
- [3] Conte Thomas M, Banerjia Sanjeev, Larin Sergei Y, Menezes Kishore N, Sathaye Sumedh W. Instruction fetch mechanisms for VLIW architectures with compressed encodings. In: IEEE/ACM international symposium on microarchitecture, proceedings of the 29th annual; 1996. p. 201–11.
- [4] <http://www.ti.com/sc/docs/psheets/rel_dsp.htm>; January 2006.
- [5] Shen John Paul, Lipasti Mikko. Modern processor design fundamentals of superscalar processors. McGraw-Hill; 2005.
- [6] Hennessy John L, Patterson David A. Computer architecture a quantitative approach. 3rd ed. Publishers: Morgan Kaufman; 2003.
- [7] Chen OT-C, Chen L-H, Lin N-W, Chen C-C. Application-specific data path for highly efficient computation of multistandard video codes. IEEE Trans Circuits Syst Video Technol 2007;17(1).
- [8] Seshan Nat. High VelocityTI processing [Texas Instruments VLIW DSP Architecture]. IEEE Signal Process Mag 1998;15(2):86–101.
- [9] Dongkun Shin, Jihong Kim, Naehyuck Chang. An operation rearrangement technique for power optimization in VLIW Instruction Fetch. In: IEEE international conference on computer design; March 2001. p. 809.
- [10] Leupers R. Instruction scheduling for clustered VLIW DSPs. In: International conference on parallel architectures and compilation techniques; October 2000. p. 291–300.
- [11] Tay-Jyi Lin, Chin-Chi Chang, Chen-Chia Lee, Chein-Wei Jen. An efficient VLIW DSP architecture for baseband processing. In: IEEE international conference on computer design; October 2003. p. 307–12.

- [12] Wanhammar Lar. DSP integrated circuits. Academic press; 1999.
- [13] Lewis M, Brackenbury L. An instruction buffer for a low-power DSP. *Adv Res Asynchronous Circuits Syst* 2000(April):176–86.
- [14] Panis C, Grunbacher H, Nurmi J. A scalable instruction buffer and align unit for xDSPcore. *IEEE J Solid-State Circuits* 2004;39(7).
- [15] Guthaus MR, Ringenberg JS, Ernst D, Austin TM, Mudge T, Brown RB. MiBench: a free, commercially representative embedded benchmark suite WWC-4. In: *IEEE international workshop*; 2001. p. 3–14.
- [16] TMS320C64x DSP Library Programmer's Reference. Texas Instruments Inc.; April 2002.
- [17] TMS320C64x Image/Video Library Programmer's Reference. Texas Instruments Inc.; April 2002.
- [18] Tay-Jyi Lin, Chie-Min Chao, Chia-Hsien Liu, Pi-Chen Hsiao, Shin-Kai Chen, Li-Chun Lin, Chih-Wei Liu, Chein-Wei Jen. Computer architecture: a unified processor architecture for RISC and VLIW DSP. In: *Proceedings of the 15th ACM great lakes symposium on VLSI*; April 2005.